

Demo 视频录制剧本（5 分钟评委导向版）

一句话目标

用 5 分钟视频向评委清晰证明：ClosedLoop 不是一个只会聊天推荐的助手，而是一个能把“用户一句话需求”推进到“可执行方案、可控调整、异常补救、Mock 支付与执行结果”的执行型本地生活 Agent。

使用说明

- 本文档面向 Demo 视频录制，不是纯复现手册。
- 建议录制方式：真实操作录屏 + 后期加速 Docker 启动过程 + 保留逐句口播。
- 视频基调：先讲后演，先把你对评分标准的理解讲给评委，再进入正式演示。
- 本项目所有执行均为 Mock：不会真实下单、不会真实扣款、不会真实调用外部业务系统。

视频总节奏

建议总时长控制在 5 分钟 左右，分配如下：

1. 开场与评分理解：35 秒 2. 项目范围与提交材料：20 秒 3. Docker 启动与工程可信度：20 秒 4. 生成首版方案 A：55 秒 5. 执行中的异常与候选修复：65 秒 6. 支付、执行结果与分享：40 秒 7. 结尾总结：15 秒

录制前准备

1. 环境准备

- 推荐使用 Docker 方式运行，录制时执行：

```
docker compose up -d --build
```

- 若只做开发调试，可本地分别启动主服务与规划子服务。
- 录制前建议先确认前端、主服务、规划子服务都能正常访问，避免视频里出现长时间等待。

2. 录屏准备

- 浏览器建议提前打开项目首页与前端页面。
- 终端字号适当放大，保证 docker compose up -d --build 可被看清。
- 若使用剪辑软件加速启动过程，建议保留“命令输入”和“服务启动完成”两个关键画面。

3. 故障注入准备

为了稳定展示 fixup / 候选修复能力，建议提前准备一个可复现的“无座或容量告急”场景。

推荐方式：在 mock_db/reservations.json 中，将目标晚餐时段的 capacity_remaining 改为 0。

参考示例：

```
{
  "target_type": "combo",
  "target_id": "combo_xxx",
  "time_slots": [
    {
      "start_time": "17:00",
      "end_time": "19:00",
      "capacity_total": 30,
      "capacity_remaining": 0,
      "queue_required": true,
      "wait_minutes": 90
    }
  ]
}
```

```
}  
]  
}
```

说明:

- capacity_remaining = 0 表示目标时段满员或无座。
- queue_required / wait_minutes 可用于增强排队拟真感，但不是必须。

正式口播稿

第一段：开场与评分理解

目标

先用最短时间把评委注意力对齐到“执行闭环”上，而不是一开始就让评委把它理解成普通推荐助手。

建议时长

35 秒

画面

- 打开 README.md
- 快速展示“创新性 / 完整性 / 应用效果 / 商业价值”四个维度
- 不细读正文，停留 3 到 5 秒即可

口播稿

大家好，我是 ClosedLoop 团队的钱惊尘。接下来由我来展示我们的项目 ClosedLoop。

它的定位不是一个只会给建议的聊天助手，而是一个执行型本地生活 Agent。

正式开始之前，我先用 README 里的四个维度对齐一下这次演示的观察重点，也就是创新性、完整性、应用效果和商业价值。

这里也欢迎评委老师在这一页稍微停一下，看一下我们希望重点证明的几个方向。

我理解这道题评委真正关心的，不只是它能不能生成一个看起来不错的方案，而是三件事。

第一，它能不能把用户一句自然语言需求，变成一个完整、时间自洽、4 到 6 小时的可执行行程。

第二，这个方案在用户确认之后，能不能继续往下执行，而不是停留在文本建议。

第三，当执行中遇到真实世界里常见的问题，比如无座、容量不足、需要替换时，它能不能低打扰地继续把事情做完。

所以接下来这 5 分钟里，我不会逐条念材料，而是直接围绕创新性、完整性、应用效果和商业价值这四个维度，用一条真实链路去证明我们的系统到底做到了什么。

第二段：项目范围与提交材料

目标

明确当前 Agent 系统支持的边界，同时顺手建立工程可信度。

建议时长

25 秒

画面

- 展示 README.md

- 展示 competition_submission/01_demo.md
- 展示 competition_submission/02_tools.md
- 展示 competition_submission/03_design.md
- 只切画面，不停留细读

口播稿

在正式演示之前，我先用十几秒说明一下当前系统支持的范围。

ClosedLoop 当前聚焦的是本地生活聚会场景下的闭环链路。用户给出一句需求之后，系统会完成意图理解、结构化约束抽取、候选召回与排序、行程规划，以及后续的局部替换和 Mock 执行。

从当前实现来看，它主要覆盖 family 和 friends 两类常见群体，适合 4 到 6 小时窗口下的常见预算、距离、饮食和活动偏好组合。

其中一部分约束是显式离散支持的，比如群体类型、距离偏好、出行方式、排队偏好和是否包含礼品；像饮食偏好、活动偏好这类信息，系统会优先按当前支持的常见标签去理解，超出部分再保留原词继续处理。

这些约束边界和支持范围，我也在设计文档里做了更细的展开，视频里这里就先说明最核心的结论。

所以它不是一个无边界的开放域万能 Agent。约束过强或者候选不足时，系统会明确告诉你当前无解或者需要放宽条件，而不是伪造一个看起来合理的结果。

这里也提前说明一下，当前所有执行都是 Mock，不会真实扣款，也不会真实对外下单。

这次提交里，我把项目说明、工具能力说明和设计说明都整理成了独立文档。视频里我只展示它们的存在，不展开逐页讲解，因为今天的重点是把整条链路真实走通。

这一段主要想说明，这不是一个临时拼出来的 prompt demo，而是一个边界清晰、有配套材料、可以复现的系统化方案。

第三段：Docker 启动与工程可信度

目标

证明系统是可启动、可运行、可复现的，而不是静态页面或预制结果。

建议时长

20 秒

画面

- 切到终端
- 输入并执行：

```
docker compose up -d --build
```

- 后期可对等待过程加速
- 最终停留在服务启动成功的画面

口播稿

接下来我用 Docker 一键启动整个系统。

这里我保留启动过程，是想证明这套演示不是提前剪好的静态结果，而是可以现场拉起、真实运行的完整系统。

当前系统是多服务协同架构，我这里重点讲两个核心的后端服务：主 Agent 服务负责对话状态和总控逻辑，规划子服务负责候选检索、过滤、排序和路线规划。

所以这个项目不是单次 prompt 直接吐一个 itinerary，而是把重逻辑拆进了明确的工程 workflow 里。

服务启动完成之后，我会直接进入前端，开始一次完整的用户闭环演示。

这一段主要想说明，我们不是只做了一个会说话的 Agent，而是做了一个可运行、可拆分、可解释、可复现的工程化系统。

第四段：生成首版方案 A

目标

先证明系统能从自然语言生成一个完整的、可继续执行的方案 A。

建议时长

55 秒

画面

- 打开前端页面
- 输入自然语言需求
- 等待聊天区摘要与方案卡片出现

建议输入

今天下午我们一家三口（2 个大人 + 1 个 5 岁小朋友）想出去玩，时间 13:00-19:00，预算 600 元以内。不吃海鲜，想要亲子友好。



口播稿

现在我先发起一个真实的用户请求。

这里我直接使用前端首页里“一家三口”这个入口对应的真实提示词，保证演示输入和前端体验是完全对齐的。

这个输入我故意保持自然语言，而不是结构化表单，因为我想证明系统第一步具备的是对真实用户表达的理解能力。同时这个例子也落在当前系统比较稳定的支持范围内。比如一家三口会落到当前支持的 family 群体类型里，4 到 6 小时和预算约束都比较明确，亲子友好、少走路、室内为主、不吃海鲜这些偏好，也能对应到当前这版系统重点覆盖的约束组合。

也就是说，我这里不是在用一条只适合演示的固定样例，而是在用一个真实、并且落在当前系统支持范围内的自然请求来做演示。后面如果评委老师自己试用，也可以直接从前端的家庭或朋友快捷提示词出发，再在原提示词上做一点微调，这样最容易体验到当前系统真正稳定的能力边界。

现在系统开始生成方案。这里大家可以看到，它不是只回复一句推荐话术，而是在聊天区给摘要，同时在方案卡片区域给出结构化结果。

这个方案里包含活动、晚餐和后续安排，每个步骤都有时间段、地点、时长，以及后续是否涉及预约或者排队的执行属性。

到这里，我想证明的是第一件事：它已经不是在告诉你“可以去哪”，而是在输出一个可以继续往下执行的方案 A。

这一段更主要对应完整性和应用效果，因为系统不仅理解了自然语言，还把结果稳定落成了一个用户能看、也能继续执行的结构化方案。

第五段：执行中的异常与候选修复

目标

证明系统在执行阶段遇到问题时，不是简单报错退出，而是能把用户继续带回可完成路径。

建议时长

65 秒

画面

- 在首版方案 A 上直接发起执行
- 触发已准备好的“无座 / 容量告急”场景
- 展示界面出现候选 1 / 候选 2
- 直接选择候选 1

建议输入

就按这个方案执行。

若系统进入 fixup 候选选择，可直接选择：

选候选1

口播稿

接下来我直接开始执行当前方案。

这里的重点就从规划切换到执行了。系统会逐步展示当前在做什么，比如预约、排队、通知，或者进入待支付状态。如果用户在执行前还想继续微调，这里其实也支持按偏好继续调整，或者通过 search 和 adjust 做局部替换。不过这次视频里，我优先展示更关键的执行闭环。

为了稳定演示异常处理能力，我这里通过 .env 里的强制失败配置，预先打开了一个可复现的告急场景。也就是说，这次执行不会一路顺滑到底，而是会在关键步骤上遇到真实世界里常见的库存或预约失败。

现在我们看到，系统已经识别到当前目标不可继续执行，并给出了候选修复方案。

这里我要特别说明一下，系统其实支持两种路径。第一种是继续 search，搜更多替代候选；第二种是直接从当前候选里做选择。

为了让视频节奏更紧凑，我这里直接选择候选 1。

这个步骤想证明的是，系统遇到失败时不是简单报错退出，而是能把用户继续带回可完成路径上。

如果用评分视角来看，这一段最核心对应的就是完整性，因为真正的执行型 Agent 不是第一次给出答案就结束了，而是在失败出现之后还能继续把事情做完。

第六段：支付、执行结果与分享

目标

把整条链路真正收尾，证明它不是“会规划”，而是“能执行到结果”。

建议时长

40 秒

画面

- 展示 pending_payment
- 输入 Mock 支付密码
- 展示执行结果摘要
- 展示分享文案或分享功能入口

建议输入

若界面进入待支付状态，输入 Mock 支付密码：

111111

口播稿

候选确认之后，系统会继续推进到支付和执行阶段。

这里我也说明一下，当前项目为了保证演示安全和可复现，支付和执行都采用 Mock 方式，不会产生真实扣款。现在界面已经进入待支付状态。我输入 Mock 支付密码之后，系统继续提交执行。

到这里，整个链路已经从一句自然语言需求，走到了方案确认、局部调整、异常修复、支付和执行结果回传。

最后我再展示一下分享能力。系统可以把最终方案整理成一段适合分享给朋友的内容，这一点在聚会场景里非常自然，因为真实用户通常不是自己一个人消费这个结果，而是需要把安排同步给同行的人。

这一段更对应应用效果和商业价值，因为结果不再停留在建议层，而是自然连接到了执行、决策同步和后续转化。

第七段：结尾总结

目标

用一句收束话把评委的注意力重新拉回评分点。

建议时长

15 秒

画面

- 停留在执行结果页或分享页

口播稿

最后我用一句话总结 ClosedLoop。

它的重点不是让模型生成一段看起来聪明的话，而是把本地生活里原本需要用户自己反复比较、调整、确认和补救的过程，收敛成一条真正可以走完的闭环链路。

如果用四个评分维度来收束的话，创新性体现在 workflow 和 agent 的工程化融合，完整性体现在从规划到执行再到异常补救的闭环能力，应用效果体现在结构化方案和流式交互体验，商业价值体现在它把一次建议真正推进到了可执行、可分享、可转化的链路里。

这就是我这次项目演示的全部内容。

录制时的关键观察点

评委最容易感知、你也最应该主动点出来的观察点如下：

- 规划质量：是否把一句自然语言真正落成了 4–6 小时窗口内的完整链路。
 - 结构化结果：是否不是只给聊天文本，而是有可消费的方案卡片和步骤信息。
 - 调整能力：是否可以围绕当前方案继续改约束、改偏好，而不是每次推翻重来。
 - 执行能力：是否在确认后进入真实的 Mock 执行流程，而不是停留在建议层。
 - 异常覆盖：是否在无座或容量不足时给出补救，而不是简单报错。
 - 低打扰：是否尽量自己做决策，只有必要时才让用户选候选。
 - 工程可信度：是否通过 Docker、服务拆分、Mock 支付与结构化材料证明系统是可复现、可解释的。
-

备用台词

1. 如果 Docker 启动稍慢

这里我保留等待过程，是想让大家看到这是实时启动的完整系统，而不是提前准备好的静态页面。

2. 如果方案生成稍慢

这里我保留一点等待时间，是想证明这个结果不是预设文案，而是系统正在实时完成约束理解和方案生成。

3. 如果 fixup 切换稍慢

这里系统正在把替换后的结果重新落回整体行程，而不是只替换一行文字，所以会有一个短暂的重算过程。

4. 如果支付过程稍慢

这里进入的是待支付后的 Mock commit 阶段，所以会比纯文本返回多一步执行确认。

常见问题

- 现象：主服务报“连接失败”或“超时”
- 处理：确认规划子服务也已启动，两个服务都需要就绪。
- 现象：首次启动较慢
- 处理：等待服务启动完成后再开始演示，剪辑时可对等待过程加速。
- 现象：fixup 没有稳定触发
- 处理：提前检查 mock_db/reservations.json 中目标时段的容量配置，确保无座场景能稳定复现。